

Docker: Intermediate

1. What are Docker namespaces and cgroups?

Great question! This one is **core Docker internals** and often asked at **medium-to-advanced level interviews**. Let's break it down clearly:

◆ Docker Namespaces

Namespaces are a **Linux kernel feature** that Docker uses to provide **isolation** for containers.

They make each container think it has its **own resources** (like its own filesystem, network, process tree).

Key namespaces Docker uses:

Namespace	Purpose
PID (Process ID)	Gives each container its own process tree . A process in one container cannot see or kill processes in another.
NET (Network)	Each container gets its own network stack (interfaces, IP addresses, routing).
IPC (Inter-Process Communication)	Isolates System V IPC and POSIX message queues.
UTS (UNIX Timesharing System)	Lets containers have their own hostname and domain name .
MNT (Mount)	Isolates mount points so containers see only their own filesystem.
USER	Maps container user IDs to different host user IDs for better security.

◆ **In short:** Namespaces = *"What resources a container can see."*

◆ Docker cgroups (Control Groups)

cgroups are another **Linux kernel feature** that Docker uses to **limit and manage resource usage** for containers.

They control:

- **CPU** usage limits
- **Memory** allocation and limits
- **Disk I/O** throttling
- **Network bandwidth** restrictions

◆ **In short:** cgroups = *"How much of a resource a container can use."*

◆ **Analogy**

Think of **Namespaces** as separate **rooms** (containers don't see other rooms), and **cgroups** as **rules** for how much electricity, water, or gas each room can use.

◆ **Quick Interview Answer**

Docker uses
Linux namespaces
cgroups

Would you also like me to add **1-2 practical commands** to inspect namespaces and cgroups on your system for interviews?

2. How do you optimize the size of a Docker image?

Here's a **complete, interview-ready explanation** on optimizing Docker image size:

◆ **Why Optimize Docker Image Size?**

- Smaller images **build faster, transfer faster, and consume less storage.**
- Reduces **attack surface** for security.

- Improves **CI/CD pipeline speed**.

◆ **Strategies to Reduce Docker Image Size**

1. **Use Minimal Base Images**

- Prefer lightweight images like alpine, distroless, or scratch instead of ubuntu or debian.

```
FROM alpine:3.18
```

1.

2. **Multi-Stage Builds**

- Build your app in one stage and copy only the final artifact to a smaller runtime image.

```
# Stage 1: build
FROM maven:3.9.1-openjdk-21 AS build
WORKDIR /app
COPY . .
RUN mvn clean package

# Stage 2: runtime
FROM openjdk:21-jdk
WORKDIR /app
COPY --from=build /app/target/myapp.jar .
CMD ["java", "-jar", "myapp.jar"]
```

1.

2. **Remove Unnecessary Files**

- Delete temp files, logs, caches, or documentation not needed in the final image.

```
RUN rm -rf /var/lib/apt/lists/*
```

1.

2. **Combine RUN Commands**

- Minimize layers by chaining commands with &&.

```
RUN apt-get update && apt-get install -y curl \  
&& rm -rf /var/lib/apt/lists/*
```

1.

2. Use .dockerignore

- Exclude files/folders like .git, node_modules, or local configs.

```
.git  
node_modules  
*.log
```

1.

2. Use Specific Versions

- Pin base images and dependencies to avoid unnecessary large upgrades.

3. Minimize Installed Packages

- Only install packages that your application actually needs.

4. Use Compressed Artifacts

- Copy pre-built .jar, .war, or binaries instead of source code + build tools.

◆ Extra Tips

- **Check image size:**

```
docker images
```

-

- **Analyze layers:**

```
docker history <image_name>
```

- - **Use tools like `dive`** to visualize and reduce image layers.
-

◆ **Quick Interview Answer**

"To optimize Docker image size, use
lightweight base images
multi-stage builds

I can also create a **cheat-sheet of 10+ practical Docker optimization tricks** that you can memorize for interviews.

Do you want me to do that?

3. Explain the multi-stage build in Docker.

Here's a **clear, interview-ready explanation** of Docker **multi-stage builds**:

◆ **What is a Multi-Stage Build?**

A **multi-stage build** allows you to use **multiple FROM statements** in a single Dockerfile.

Each stage can have its own **base image and commands**, and you can **copy artifacts** from one stage to another.

Purpose:

- **Reduce final image size** by copying only the necessary artifacts (e.g., compiled binaries) to the final stage.
 - Keep **build tools and dependencies out** of the final runtime image.
 - Simplify Dockerfiles for complex builds.
-

◆ **How It Works**

1. **Stage 1: Build Stage**

- Use a larger image with all build tools (e.g., Maven, Gradle, Node.js).
- Compile or build your application.

2. Stage 2: Runtime Stage

- Use a minimal base image (e.g., openjdk:21-jdk, alpine).
- Copy only the final artifact from the build stage.
- Run the application.

◆ Example: Java Spring Boot Application

```
# Stage 1: Build
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY . .
RUN ./gradlew bootJar

# Stage 2: Runtime
FROM eclipse-temurin:21-jdk
WORKDIR /app
COPY --from=build /app/build/libs/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Explanation:

- Stage 1 builds the JAR with Gradle (includes all build tools).
- Stage 2 copies only the .jar file into a clean image, leaving behind Gradle, source code, and caches.
- Result: **much smaller image** with only what's needed to run the app.

◆ Benefits

- ✓ Smaller image size → faster pulls & deployments
- ✓ Security → build tools aren't in production image
- ✓ Cleaner images → only necessary files included
- ✓ Flexibility → multiple stages for tests, builds, and runtime

◆ Quick Interview Answer

“A multi-stage build in Docker uses multiple FROM statements to separate

build

runtime

If you want, I can also create a **diagram showing multi-stage build flow**, which is super helpful for interviews.

Do you want me to do that?

4. How would you handle environment variables in Docker?

Here's a **comprehensive, interview-ready explanation** for handling environment variables in Docker:

◆ Why Environment Variables in Docker?

- Allow **configurable containers** without hardcoding values.
- Useful for **database URLs, API keys, credentials, and service endpoints**.
- Helps **differentiate between environments** (development, staging, production).

◆ Ways to Handle Environment Variables in Docker

1 Using **-e** or **--env** in docker run

```
docker run -e "DB_HOST=localhost" -e "DB_PORT=5432" my-image
```

- Simple, one-off variables when running a container.

2 Using **--env-file**

```
# .env
DB_HOST=localhost
DB_PORT=5432
API_KEY=secret123
```

```
docker run --env-file .env my-image
```

- Keeps variables **organized and version-controlled**.

3 Define in docker-compose.yml

```
version: '3.8'
services:
  app:
    image: my-image
    environment:
      DB_HOST: localhost
      DB_PORT: 5432
      API_KEY: secret123
```

- Works well for **multi-container applications**.

4 Use ARG and ENV in Dockerfile

```
# build-time argument
ARG JAR_FILE=app.jar
# environment variable inside container
ENV JAVA_OPTS="-Xmx512m"
```

- ARG → available **only during build**
- ENV → available **inside running container**

5 Combine with Spring Boot Profiles

- Example in application-docker.yml:

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASS}
```


- Set DB_URL, DB_USER, DB_PASS via Docker environment variables.
 - Allows **automatic config switching between local and docker profiles**.
-

◆ **Best Practices**

- Use .env files for **sensitive configs** and keep them out of version control (.gitignore).
 - Avoid hardcoding secrets → consider **Docker secrets** or **Vault** for production.
 - Keep environment variables **consistent across services** in Docker Compose.
 - Prefer ENV in Dockerfile for **defaults**, override them with docker-compose or docker run.
-

◆ **Quick Interview Answer**

"In Docker, environment variables can be handled via -e or --env in docker run, using an .env file, or in docker-compose.yml under environment. Inside the Dockerfile, we can use ENV for defaults and ARG for build-time variables. This allows configurable, secure, and portable containers across environments."

If you want, I can **make a full Docker interview cheat sheet** covering **all commands, optimizations, networking, volumes, multi-stage builds, and env vars** in one page for quick prep.

Do you want me to do that?

5. What are Docker Compose and its benefits?

Here's a **clear, interview-ready explanation** of Docker Compose:

◆ **What is Docker Compose?**

Docker Compose is a **tool for defining and running multi-container Docker applications**.

- It uses a **YAML file (docker-compose.yml)** to configure services, networks, and volumes.
- Instead of manually running multiple docker run commands, you can start **all containers with a single command**.

◆ Key Features

1. **Multi-container orchestration** – define all services in one file.
2. **Service dependencies** – specify startup order using depends_on.
3. **Networking** – automatically creates a network for containers to communicate.
4. **Volume management** – attach persistent storage easily.
5. **Environment variables** – configure services for different environments.
6. **Profiles** – run different sets of services for dev, staging, or production.

◆ Basic Example

```
version: '3.8'

services:
  web:
    image: my-web-app
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
    depends_on:
      - db

  db:
    image: postgres:14
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
```

```
volumes:  
  - db-data:/var/lib/postgresql/data
```

```
volumes:  
db-data:
```

- **Start all services:**

```
docker-compose up
```

- **Stop all services:**

```
docker-compose down
```

◆ Benefits of Docker Compose

1. **Simplifies multi-container setup** – manage all containers in one file.
2. **Easy to reproduce environments** – dev, staging, prod are consistent.
3. **Supports scaling services** – e.g., `docker-compose up --scale web=3`.
4. **Improved maintainability** – declarative configuration, easy to version control.
5. **Integrated networking & volumes** – containers communicate automatically.
6. **Speeds up CI/CD pipelines** – start complex environments with one command.

◆ Quick Interview Answer

“Docker Compose is a tool to define and manage multi-container Docker applications using a single YAML file. It simplifies service orchestration, networking, volumes, and environment configuration, making it easier to develop, test, and deploy complex containerized applications.”

If you want, I can **also make a list of common Docker Compose commands and tips** that interviewers frequently ask.

Do you want me to do that?

6. How do you configure networking between multiple containers?

Here's a **comprehensive, interview-ready explanation** on Docker networking between containers:

◆ Docker Networking Basics

Docker creates **virtual networks** to allow containers to communicate:

1. **Bridge (default)** – containers on the same host can communicate via container name or IP.
2. **Host** – container shares the host's network stack (no isolation).
3. **None** – container has no network.
4. **Overlay** – spans multiple Docker hosts (used in Swarm/Kubernetes).

By default, Docker Compose creates a **bridge network** for all services in a docker-compose.yml.

◆ How to Connect Multiple Containers

1 Using Docker Compose

- Containers in the same Compose file are automatically in the same network.
- They can communicate via **service name**.

Example:

```
version: '3.8'

services:
  web:
    image: my-web-app
    ports:
      - "8080:8080"
    environment:
      - DB_HOST=db
```

```
depends_on:
  - db

db:
  image: postgres:14
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: pass
```

- web container can reach db using **hostname db**.
- Docker Compose creates a network named <project>_default by default.

2 Custom Network

```
networks:
  app-network:

services:
  web:
    image: my-web-app
    networks:
      - app-network

  db:
    image: postgres:14
    networks:
      - app-network
```

- Explicit network gives **better control**, can span multiple Compose files or external containers.

3 Docker CLI Networking

- Create a network:

```
docker network create app-network
```

- Run containers in that network:

```
docker run -d --name web --network app-network my-web-app
docker run -d --name db --network app-network postgres:14
```

- Now web can communicate with db using **container name** as hostname.

◆ Networking Tips

1. **Service discovery** – use **container/service names** instead of hardcoded IPs.
2. **Expose only necessary ports** – internal communication doesn't need host ports.
3. **Use bridge network for isolation** – keep unrelated services in separate networks.
4. **Overlay networks** for multi-host communication in production.

◆ Quick Interview Answer

"In Docker, containers can communicate over virtual networks. By default, Docker Compose creates a bridge network allowing containers to talk to each other via service names. You can also create custom networks to control isolation and communication. Overlay networks are used for multi-host setups."

If you want, I can **make a full table of Docker network types with pros, cons, and common commands** for interview prep.

Do you want me to do that?

7. Explain bind mounts vs. volumes.

Here's a **clear, interview-ready explanation of bind mounts vs. volumes in Docker**:

◆ Why Mounts Are Needed

Containers have **ephemeral filesystems**—any data inside disappears when the container stops.

Mounts allow containers to **persist data** outside the container.

◆ 1 Bind Mounts

- Maps a **specific host directory or file** directly into the container.
- Docker **does not manage** the lifecycle—what exists on the host is what the container sees.
- **Path is absolute on host.**

Example:

```
docker run -v /home/user/app/config:/app/config my-image
```

- Host `/home/user/app/config` → Container `/app/config`
- Any changes on host reflect immediately in the container and vice versa.

Pros:

- Useful for **development**: live code changes reflected in container.
- Full control of host files.

Cons:

- Depends on host filesystem structure → less portable.
 - Security risk if container modifies critical host files.
-

◆ 2 Volumes

- **Managed by Docker**, stored in Docker's storage area (`/var/lib/docker/volumes/`).
- Easier to **backup, migrate, and share** across containers.
- Lifecycle is **decoupled from host filesystem**.

Example:

```
docker volume create my-data
docker run -v my-data:/app/data my-image
```

- Docker manages my-data internally.
- Can attach the same volume to multiple containers.

Pros:

- Portable and isolated from host paths.
- Easier for production use.
- Can be backed up or mounted in different containers.

Cons:

- Not as convenient for live development edits as bind mounts.

◆ Comparison Table

Feature	Bind Mount	Volume
Storage Location	Host filesystem	Docker-managed
Portability	Low	High
Lifecycle	Manual	Managed by Docker
Use Case	Development, live code	Production, persistent data

◆ Quick Interview Answer

"Bind mounts map a specific host path directly into a container, useful for development but less portable. Volumes are Docker-managed storage, isolated from host paths, easier to backup, share, and ideal for production data persistence."

If you want, I can **make a full Docker interview question set** covering **containers, images, networking, mounts, volumes, Docker Compose, multi-stage builds, and optimizations** all in one page for rapid prep.

Do you want me to do that?

8. How to persist data in Docker containers?

Here's a **complete, interview-ready explanation** for persisting data in Docker containers:

◆ Why Data Persistence is Important

Containers are **ephemeral**:

- Any data written inside a container's filesystem is **lost when the container stops or is removed**.
 - For databases, logs, or uploaded files, persistence is essential.
-

◆ Ways to Persist Data in Docker

1 Volumes (Recommended)

- Docker-managed storage, stored in `/var/lib/docker/volumes/` by default.
- Can be shared across multiple containers.
- Lifecycle is managed by Docker.

Example:

```
docker volume create db-data
docker run -d -v db-data:/var/lib/postgresql/data postgres:14
```

- `db-data` volume persists database files even if the container is removed.

In Docker Compose:

```
services:
  db:
    image: postgres:14
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

2 Bind Mounts

- Maps a **host directory or file** directly into the container.

- Useful for **development**, live edits, or sharing files.

Example:

```
docker run -v /home/user/app/data:/app/data my-app
```

- Changes in /home/user/app/data reflect in the container and persist beyond container lifecycle.

3 tmpfs Mounts

- Stores data in **memory (RAM)** rather than disk.
- Not persistent across container restarts, but fast and isolated.

```
docker run -d --tmpfs /app/tmp my-app
```

- Useful for temporary or sensitive data.

4 Database or External Storage

- Connect containerized apps to **external databases or cloud storage** for persistent data.
- Example: AWS S3, Azure Blob, or a managed Postgres outside Docker.

◆ Best Practices

1. **Use volumes for production** databases and critical data.
2. **Bind mounts** for development and live code updates.
3. Avoid storing persistent data in the container's **writable layer**.
4. Backup and version control volumes if necessary.

◆ Quick Interview Answer

"To persist data in Docker, use
volumes
bind mounts

If you want, I can also **make a visual diagram showing bind mounts vs. volumes vs. tmpfs** for easy memorization before interviews.

Do you want me to do that?

9. How do you troubleshoot a failing Docker container?

Here's a **comprehensive, interview-ready guide** on troubleshooting a failing Docker container:

◆ Step 1: Check Container Status

```
docker ps -a
```

- Verify if the container is running, exited, or restarting.
- Look at **Exit Code** (e.g., 1, 137) for hints:
 - 1 → generic error
 - 137 → killed by OOM (out of memory)
 - 139 → segmentation fault

◆ Step 2: Inspect Logs

```
docker logs <container_name_or_id>
```

- Check for **error messages** or **stack traces**.
- Use -f to **follow logs in real-time**:

```
docker logs -f my-container
```

◆ Step 3: Inspect Container

```
docker inspect <container_name_or_id>
```

- Shows configuration, environment variables, mount points, network info.
- Useful for checking **volume mounts, port bindings, and environment variables**.

◆ Step 4: Access the Container

```
docker exec -it <container_name_or_id> /bin/bash  
# or sh if bash is not available
```

- Check filesystem, logs, running processes, and configuration files inside container.

◆ Step 5: Check Resource Limits

- Containers may fail due to **insufficient memory or CPU**.

```
docker stats
```

- Adjust `--memory` or `--cpus` if needed.

◆ Step 6: Verify Networking

- If container depends on other services (e.g., DB, Kafka):

```
docker network ls  
docker network inspect <network_name>  
docker exec -it <container_name> ping <service_name>
```

- Ensure **hostnames, ports, and network connectivity** are correct.

◆ Step 7: Check Image & Build

- Verify the **Dockerfile and image**:

```
docker build .  
docker run <image_name> /bin/bash
```

- Errors may come from **missing dependencies, wrong working directory, or incorrect commands** in Dockerfile.

◆ Step 8: Remove & Recreate Container

- Sometimes a fresh start fixes issues:

```
docker rm -f <container_name>
docker run ...
```

- Use `--force-recreate` in **docker-compose**:

```
docker-compose up --build --force-recreate
```

◆ Step 9: Enable Debugging

- For Spring Boot apps:

```
spring:
  profiles:
    active: docker
  logging:
    level:
      root: DEBUG
```

- Allows seeing **stack traces and initialization errors**.

◆ Quick Interview Answer

“To troubleshoot a failing Docker container, first check its status with `docker ps -a`, inspect logs using `docker logs`, and explore the container with `docker exec`. Verify environment variables, volume mounts, ports, and resource limits. If needed, rebuild the image or recreate the container, and enable application-level debugging for more details.”

If you want, I can **create a full Docker troubleshooting checklist** for interviews that covers **logs, inspect, exec, network, volumes, resource limits, and rebuild steps** in one page.

Do you want me to do that?

10. What is the difference between docker-compose up and docker-compose up --build?

Here's a **clear, interview-ready explanation** of the difference:

◆ **docker-compose up**

- Starts the services defined in docker-compose.yml.
- If the images for services **already exist**, it uses them **without rebuilding**.
- Creates and starts containers for all services in the Compose file.
- If containers already exist, it **reuses them** unless you specify --force-recreate.

Example:

```
docker-compose up
```

- Starts services using existing images and containers.
- Faster if images haven't changed.

◆ **docker-compose up --build**

- **Rebuilds images** before starting containers.
- Useful when:
 - You **changed the Dockerfile**.
 - You **updated application code** in a COPY or ADD step.
- Ensures the latest changes are included in the running containers.

Example:

```
docker-compose up --build
```

- Forces a rebuild of all service images, then starts containers.

◆ Key Difference

Feature	docker-compose up	docker-compose up --build
Build images	No	Yes
Use cached image	Yes	Rebuilds image even if cached exists
Use case	Starting services quickly	When Dockerfile or source code changed

◆ Quick Interview Answer

"docker-compose up starts containers using existing images, while docker-compose up --build rebuilds the images first before starting the containers. Use --build when you've changed the Dockerfile or application code."

If you want, I can also **make a full Docker vs Docker Compose cheat sheet** with commands, flags, and tips for interviews.

Do you want me to do that?